RGD-Driver: Release-Aware Gated Deliberation for Large Language Models in Closed-Loop Autonomous Driving

Liyi Xie, Chuyue Zeng, Zhonghui Yang, Bao Zhao, Jianwei Chen,
Shuang Huang, Xuan Li, and Fang Deng, *Fellow, IEEE*

***Abstract*—Large language models have demonstrated strong capabilities in semantic scene interpretation and high-level decision making for autonomous driving. However, their inference latency introduces a fundamental release-time mismatch: a reasoning request is issued from one traffic state, whereas its response becomes available only after the scene has evolved. Existing dual-process and asynchronous driving systems focus on when to invoke the slow reasoner or how to maintain fast control during inference, yet lack an explicit mechanism to determine whether a delayed proposal remains valid, distinct from the current fast action, and beneficial in the evolved state. This paper presents RGD-Driver, a two-stage framework based on Release-Aware Gated Deliberation (RGD) that governs both slow-reasoner admission and release validation in closed-loop driving. At query time, RGD issues a request only when delay feasibility, action support, recovery-cost advantage, and scene demand are jointly satisfied. At release, the returned proposal is remapped in the observed state and authorized only after passing feasibility, action-distinctness, and recovery-cost checks against the current fast command. We further introduce Recoverable Value of Deliberation (R-VoD), an offline matched-rollout diagnostic that quantifies the corrective opportunity available at release. Closed-loop experiments on highway-env and MetaDrive demonstrate that RGD substantially reduces slow-path invocations while preserving task outcomes under matched-seed evaluation, and that release validation retains the corrective intervention while filtering harmful or negligible alternatives.**

***Index Terms*—Autonomous driving, closed-loop decision making, inference latency, large language models, recoverability, selective computation.**

I. Introduction

LARGE language models (LLMs) have advanced semantic scene interpretation, driving-knowledge retrieval, and high-level decision making in autonomous driving [1]–[4]. These capabilities provide a deliberative complement to reactive policies, particularly when a decision depends on scene relations, driving rules, or long-horizon intent. Closed-loop deployment, however, imposes a temporal requirement that is absent from static decision benchmarks. A reasoning request is issued from a query state, but its proposal becomes available in a later release state after the fast controller and surrounding traffic have continued to evolve. Consider a closing lead vehicle with an initially open adjacent lane. A lane-change proposal that is appropriate when requested may arrive after the target gap has narrowed, after the maneuver has lost its advantage, or after the fast controller has already selected the same action. Thus, semantic correctness at query time does not establish control authority at release time. The central problem is not simply how rapidly an LLM responds, but whether its delayed output still constitutes a valid and useful intervention in the state where it can affect the vehicle.

Existing systems address parts of this problem from different control points. Selective-computation methods determine whether a scene warrants slow reasoning before inference begins. Asynchronous architectures preserve a fast control stream while the slow branch is pending, and runtime-assurance methods test whether a candidate action satisfies an admissible safety envelope. These functions are necessary, but they answer different questions: whether to spend computation, how to maintain control during inference, and whether an action is safe in isolation. None of them establishes whether a returned proposal should replace the concurrent fast command. That decision requires the response to be remapped in the observed release state and compared with the fast branch through the same action and cost interfaces. Feasibility alone is insufficient because a safe proposal may be redundant or offer no corrective advantage. Conversely, query admission cannot grant persistent authority to an action that has not yet been generated. Delayed deliberation therefore requires two separate authority decisions: one before computation and another before actuation.

This paper presents RGD-Driver, which treats the slow reasoner as a release-conditioned proposal generator rather than a persistent command source. Release-Aware Gated Deliberation (RGD) first admits a request only when delay feasibility, alternative-action support, recovery-cost advantage, and scene demand hold jointly. When the response arrives, RGD remaps the proposal in the current state and authorizes it only if it remains feasible, differs from the current fast command, and provides a sufficient recovery-cost advantage. Rejected, malformed, or expired proposals leave the fast control stream unchanged. We further define Recoverable Value of

This work was supported in part by the National Natural Science Foundation of China under Grant 62203250, in part by Young Top-notch Talent of the Guangdong Special Support Program under Grant 2025TQ09X872, in part by the Young Elite Scientists Sponsorship Program of China Association of Science and Technology under Grant YESS20210289, in part by the China Postdoctoral Science Foundation under Grant 2020TQ1057 and Grant 2020M682823.

Liyi Xie, Chuyue Zeng, Zhonghui Yang, Bao Zhao, Jianwei Chen, Shuang Huang, and Xuan Li are with the Faculty of Marine Science and Technology, Beijing Institute of Technology, Zhuhai 519000, China (e-mail: {3220252962, 3220252976, 3120252122, baozhao, chenjianwei, huangshuang}@bit.edu.cn and lixuan@bitzh.edu.cn). Fang Deng is with the School of AI, Beijing Institute of Technology, Beijing, China (e-mail: dengfang@bit.edu.cn).

Deliberation (R-VoD), an offline matched-rollout measure of the corrective opportunity available at a release state under a common continuation policy. R-VoD is kept outside the online gate and is used to test whether the states selected by RGD retain meaningful intervention value. Paired closed-loop experiments, controlled-delay analysis, component ablations, and matched-proposal replay connect the resource-allocation result to the specific admission and release mechanisms that produce it.

The main contributions are as follows:

1) We develop RGD, a two-stage authority framework coupling conjunctive query admission with current-state release validation. A shared action mapper, recovery-cost evaluator, and fast-path fallback form a release contract that lets delayed proposals affect actuation only when feasible, distinct, and advantageous.
2) We introduce R-VoD, an offline matched-rollout measure of corrective opportunity at release by varying the first effective action under a common continuation policy. It separates release-state value from reasoner output quality and provides an independent diagnostic for mechanism analysis.
3) Closed-loop results on highway-env and MetaDrive show that RGD reduces continual slow-path use by over 90% while preserving paired task outcomes, and that release validation retains the corrective intervention while filtering harmful or negligible alternatives.

II. Related Work

A. Language-Conditioned and Generative Driving

Language-conditioned driving methods differ in both their decision role and the proximity of their outputs to vehicle actuation. At the advisory level, DiLu retrieves relevant experience before producing an interpretable decision [1], Driving with LLMs represents traffic through object-level vectors and returns actions with explanations [2], and GPT-Driver formulates planning as language modeling [3]. Driving-knowledge enhancement supplies domain-specific context before decision making [4]. These studies establish that language models can encode scene relations, retrieve driving knowledge, and express high-level intent through structured or interpretable interfaces.

Other approaches place model outputs closer to the planning and control boundary, including closed-loop end-to-end driving [5]. C-TRAIL incorporates commonsense reasoning into trajectory planning [6], while policy-transfer methods use language models to resolve task descriptions and select reusable driving policies [7]. Generative driving methods learn predictive scene representations or future trajectories for downstream planning [8]–[11]. This progression improves the operational relevance of model outputs, but it also makes temporal alignment more important: the closer a proposal is to actuation, the fewer downstream stages remain to correct an action derived from an earlier state. These studies primarily address how to produce a capable driving proposal or representation. RGD addresses the complementary systems question of how a delayed proposal acquires actuation authority. It does not replace the underlying reasoner, planner, or learned representation. Instead, it provides a common release interface that reinterprets the returned output in the current state and compares it with the action already supplied by the fast controller.

B. Dual Process Reasoning and Selective Computation

Dual-process driving systems combine the response frequency of a fast policy with the contextual reasoning of a deliberative model. FASIONAD coordinates the two branches through adaptive feedback [12], AdaDrive adapts deliberation depth to scene demand [13], NetRoller connects general and specialized models through asynchronous execution [14], and LeapAD uses reflection and accumulated experience within a dual-process architecture [15]. Their principal contribution is to determine when or how the deliberative branch should participate while a responsive policy remains available.

Selective computation is closely related to metareasoning, which balances expected decision improvement against computational cost. Time-bounded and anytime methods schedule reasoning under a finite budget [16]–[19], and concurrent planning permits an executor to continue acting while a planner computes [20]. These principles motivate query admission, but query-time value is only an estimate of a future opportunity. In closed-loop driving, traffic evolves and the fast controller continues to act during inference. The alternative that justified a request may therefore disappear, become redundant, or be superseded before the response arrives. RGD extends selective computation from invocation to authority. The admission gate controls whether slow reasoning is worth initiating, whereas the release gate controls whether its realized output is worth applying. R-VoD then measures the remaining corrective opportunity at the release state rather than the expected value inferred at the query state. This separation prevents a successful request from being treated as an unconditional future command.

C. Runtime Safety, Delay, and Viability

Timing and assurance mechanisms provide two further components of a deployable system. AsyncDriver separates language-model planning from real-time control [21], while Simplex-based architectures retain a trusted controller when a learned action fails an acceptance test [22]. Event-triggered control determines when stabilizing updates are required [23], and edge-computing frameworks schedule offloaded computation over heterogeneous resources [24]. Delay-aware control explicitly models state evolution during input lag [25]. Together, these methods address control continuity, computation timing, and delayed system dynamics.

Formal safety methods define whether an action remains inside an admissible set. RSS specifies responsibility-sensitive safety distances [26], and viability theory constructs invariant motion sets for continuous dynamics [27]. Such certificates answer whether a candidate can be executed safely, but a release decision is comparative as well as admissibility based. A delayed proposal may satisfy a safety constraint while duplicating the fast action or providing no recovery benefit.

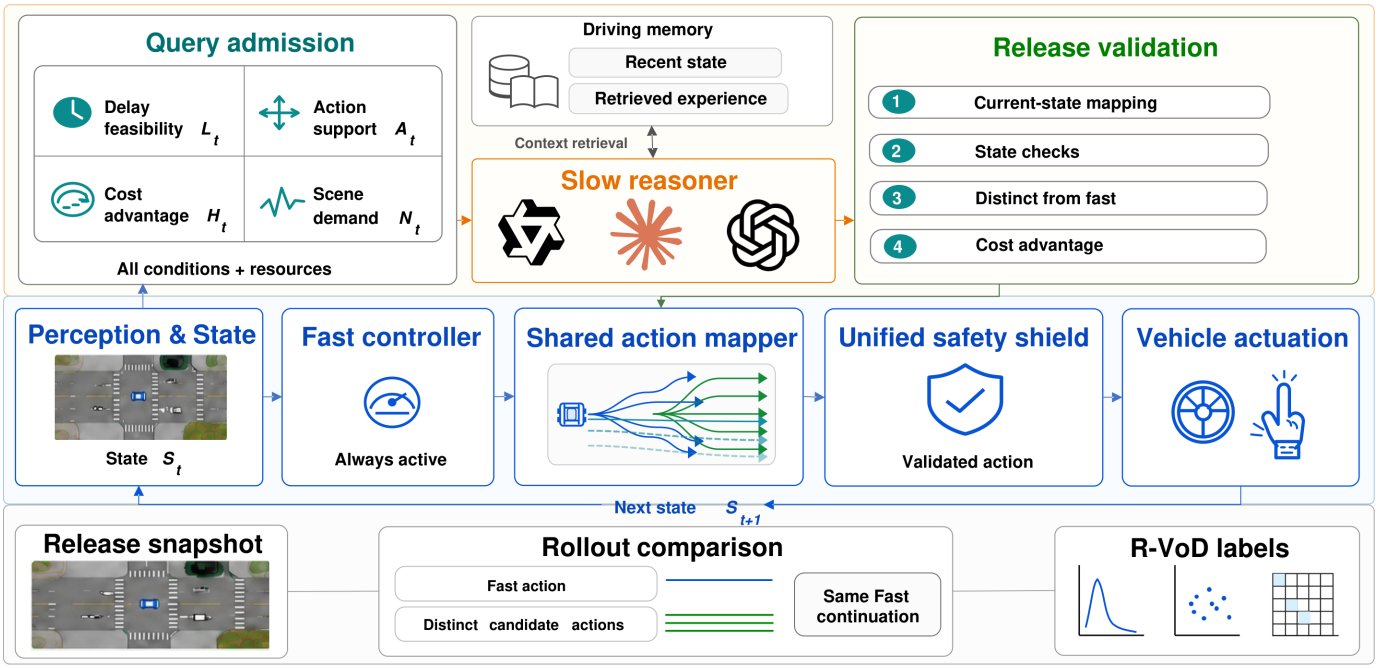

Fig. 1. RGD system architecture. Solid arrows indicate online control paths. The lower branch indicates the offline R-VoD diagnostic path. The memory module belongs to the slow-reasoner input context, not to the online gate.

RGD therefore operates before the final safety projection: it remaps both branches in the observed release state, tests feasibility, distinctness, and recovery-cost advantage, and retains the fast command whenever the slow proposal fails this comparison. The downstream safety projection remains active regardless of which branch is selected. Across these research directions, model capability, query-time allocation, asynchronous execution, and runtime safety are well studied as separate functions. The unresolved interface lies between response arrival and actuation. RGD makes this interface explicit and testable, while R-VoD provides an offline measure aligned with the release states that the online system actually encounters.

III. Release-Aware Gated Deliberation

RGD decomposes slow-reasoner invocation into two sequential authority decisions: query admission and release validation. Admission determines whether the current scene warrants deliberation and whether a feasible alternative can be supported within the expected response interval. Release validation evaluates the returned proposal against the observed state at the time of delivery. The fast controller remains the default throughout inference and is retained whenever either stage withholds slow-path authority. A release state s_r is *recoverable* if it admits at least one feasible effective alternative whose matched-rollout advantage over the fast effective action meets the R-VoD margin. R-VoD quantifies this corrective opportunity offline via matched rollouts. A returned proposal is *actionable* at release only if its mapped command passes current-state feasibility, differs from the concurrent fast command, and satisfies the proposal-specific recovery-cost margin. The final safety projection may subsequently collapse an authorized slow command to the same executable action as the fast command. R-VoD and release validation thus serve complementary, not interchangeable, roles.

Fig. 1 illustrates the two online gates and the offline R-VoD diagnostic branch. At each cycle, the observed state feeds an always-active fast controller and the query-admission gate, whose joint checks of delay feasibility L_t, action support A_t, cost advantage H_t, scene demand N_t, and available resources determine whether the slow reasoner is queried. The driving memory supplies recent states and retrieved experience as additional context for the slow reasoner. Release validation first maps and checks the slow response against the concurrent fast command. The authorized slow command then joins the fast default at the shared action mapper, before the safety shield projects the selection to actuation and closes the loop. The lower branch snapshots the release state, compares fast and candidate rollouts, and generates R-VoD labels offline for diagnosis rather than online control.

A. Query and Release Process

At frame t, the ego vehicle observes state s_t. The fast policy supplies the canonical command $a_t^f \in \mathcal{A}(s_t)$ directly to the shared mapper as the default. A slow response reaches the mapper only after release validation, where $\psi(y)$ parses it and $\mathcal{M}(s, y)$ constructs its command in $\mathcal{A}(s)$. The mapper selects the fast or authorized slow command before $\Phi(s, a)$ maps it to an executable action. Each command $a \in \mathcal{A}(s)$ contains a maneuver primitive and target speed. Two expressions are equivalent if they induce the same pair. The maneuver component comprises *lane-left*, *lane-hold*, *lane-right*, *accelerate*, and *decelerate*, all resolved through the same simulator interface. The proposal mapping is undefined when the maneuver or target speed is invalid in s. This constitutes current-state semantic mapping rather than trajectory replanning. The returned

proposal is remapped in the observed release state s_r. A lane-change or speed command that was feasible at query time may no longer be valid. Any mapping or feasibility failure causes a_r^f to be retained. The delay estimate is used solely at admission. The actual release frame r is the first control frame at which the response becomes available, and both the returned command and the fast command are evaluated in the observed state s_r. Responses that do not arrive before episode termination are discarded.

B. Query Admission

Let $\mathcal{A}(s_t)$ denote the exact runtime action domain and a_t^f the canonical fast command. The runtime recovery-cost evaluator is

$$c_t(a) = \mathrm{clip}_{[0,1]}\left(h_t + \left[d_t(a) - \min_{b\in\mathcal{A}(s_t)} d_t(b)\right]_+\right), \quad (1)$$

where $[x]_+ = \max(x, 0)$. Here, $d_t(a) \in [0,1]$ is the fixed-weight penalty combining safety, comfort, and efficiency costs, and $h_t \in [0,1]$ is the scene-level hazard aggregating TTC urgency, headway deficit, obstacle proximity, and interaction pressure. Lower values of c_t are preferred. Malformed cost inputs or inconsistent action domains suppress admission. The domain integrity guard F_t requires consistent action-domain agreement and a nonempty feasible-alternative set $\mathcal{D}_t = \{a \in \mathcal{A}(s_t) \setminus \{a_t^f\} : c_t(a) \le \kappa_c\}$, where $\kappa_c \in (0,1]$ is the recovery-cost feasibility threshold. We set $F_t = 1$ when the cost inputs are valid, the action domains agree, and $\mathcal{D}_t \neq \varnothing$, and $F_t = 0$ otherwise. If $\mathcal{D}_t = \varnothing$, the minimum in (3c) is not evaluated and no slow request is issued.

Delay feasibility is characterized by control frequency $f > 0$, predicted latency $\widehat{\tau}_t$, critical maneuver window T_t^{crit} (estimated from the current TTC and admissible trajectory corridor), and safety reserve ρ, with $T_t^{\mathrm{crit}} > 0$. The residual-window score is

$$\ell_t = \mathrm{clip}_{[0,1]}\left(\frac{T_t^{\mathrm{crit}} - \lceil f\widehat{\tau}_t\rceil/f - \rho}{T_t^{\mathrm{crit}}}\right). \quad (2)$$

Unavailable timing evidence sets $L_t = 0$. Let $\mathcal{F}$ denote the fixed set of non-Fast maneuver families, $\mathcal{F}_t \subseteq \mathcal{F}$ those represented in $\mathcal{D}_t$, u_m the minimum recovery cost among members of family m in $\mathcal{D}_t$, and $u_* = \min_{m\in\mathcal{F}_t} u_m$. Normalizing by $|\mathcal{F}|$ scores family coverage as an absolute fraction of all families, so that absent families penalize the score. If $\mathcal{F}_t = \varnothing$, then $A_t = 0$. The four admission tests are

$$L_t = \mathbb{I}[\ell_t \ge \lambda_L], \quad (3a)$$

$$A_t = \mathbb{I}\left[|\mathcal{F}|^{-1}\sum_{m\in\mathcal{F}_t} e^{-(u_m-u_*)/T_A} \ge \lambda_A\right], \quad (3b)$$

$$H_t = \mathbb{I}\left[\kappa_c^{-1}[c_t(a_t^f) - \min_{a\in\mathcal{D}_t} c_t(a)]_+ \ge \lambda_H\right], \quad (3c)$$

$$N_t = \mathbb{I}[\max(h_t, n_t^{\mathrm{pre}}) \ge \lambda_N], \quad (3d)$$

where $n_t^{\mathrm{pre}} \in [0,1]$ is an independent scene-hazard pre-screen score derived from obstacle density and conflict geometry, and $T_A > 0$ is the soft-aggregation temperature for family support. All thresholds $(\lambda_L, \lambda_A, \lambda_H, \lambda_N)$ are selected on the calibration cohort and frozen prior to the reported evaluation. Here, N_t identifies deliberation demand, whereas F_t rejects states without a feasible recovery alternative. All four admission tests must hold simultaneously.

Let $v_t \in \{0,1\}$ encode resource availability, including request budget, cooldown, and executor status, subject to the requirement that no slow request is already pending. The admission rule is

$$q_t = \mathbb{I}[v_t \wedge F_t \wedge L_t \wedge A_t \wedge H_t \wedge N_t], \quad (4)$$

where $\mathbb{I}[\cdot]$ is the indicator function.

C. Release Validation

At release frame r, let y_r denote the returned response and $\mathcal{M}$ the release-state proposal mapper used inside the gate. The fast policy independently produces a_r^f from the same state s_r. If $\mathcal{M}(s_r, y_r)$ is undefined, the response is rejected and a_r^f is retained. Otherwise, let $\widetilde{a}_r^{\mathrm{sl}} = \mathcal{M}(s_r, y_r)$ denote the mapped slow command. The domain, absolute-feasibility, maneuver-support, and cost-headroom tests represented by F_r, A_r, and H_r, together with c_r, are recomputed from s_r and the concurrent a_r^f. Delay feasibility and scene demand are admission-only conditions that do not re-enter release. The feasible slow-command set is defined as $\mathcal{V}_r = \{a \in \mathcal{A}(s_r) : c_r(a) \le \kappa_c\}$ if $F_r \wedge A_r \wedge H_r$ holds, and $\mathcal{V}_r = \varnothing$ otherwise. The release indicator is

$$g_r = \mathbb{I}\left[\widetilde{a}_r^{\mathrm{sl}} \in \mathcal{V}_r \wedge \widetilde{a}_r^{\mathrm{sl}} \neq a_r^f \wedge c_r(\widetilde{a}_r^{\mathrm{sl}}) + \delta \le c_r(a_r^f)\right], \quad (5)$$

where $\delta \ge 0$ is the release margin (expressed on the same scale as c_r), fixed at $\delta = 0.02$.

The selected and executed commands are accordingly

$$a_r^{\mathrm{sel}} = \begin{cases} \widetilde{a}_r^{\mathrm{sl}}, & g_r = 1, \\ a_r^f, & g_r = 0, \end{cases} \qquad a_r^{\mathrm{exec}} = \Phi(s_r, a_r^{\mathrm{sel}}). \quad (6)$$

Equation (6), implemented by the downstream shared mapper, selects the authorized slow command when $g_r = 1$ and the fast default otherwise. It makes the fast-path fallback explicit: whenever $g_r = 0$, $a_r^{\mathrm{exec}} = \Phi(s_r, a_r^f)$. Distinctness is evaluated in the common action domain prior to the projection Φ. Consequently, an authorized slow command may still map to the same executable action as the fast command after projection, and this post-projection outcome is reported separately. Query admission confers no persistent authority on a returned proposal, and no R-VoD label enters the online release decision. All checks operate on the observed state s_r and authorize only proposals that provide a strict recovery-cost advantage over the concurrent fast action.

D. Offline Measure of Corrective Opportunity

Offline matched rollouts provide a state-level diagnostic of the corrective opportunity available at release, independent of any specific slow-reasoner response. At each logged release snapshot s, let $a_{\mathrm{eff}}^f = \Phi(s, a^f)$ denote the effective fast first action after the common final safety projection. Let $\mathcal{D}(s)$ be the set of feasible effective first actions that remain distinct from a_{eff}^f after the same projection, and let $\mathcal{C}(s) = \{a_{\mathrm{eff}}^f\} \cup$

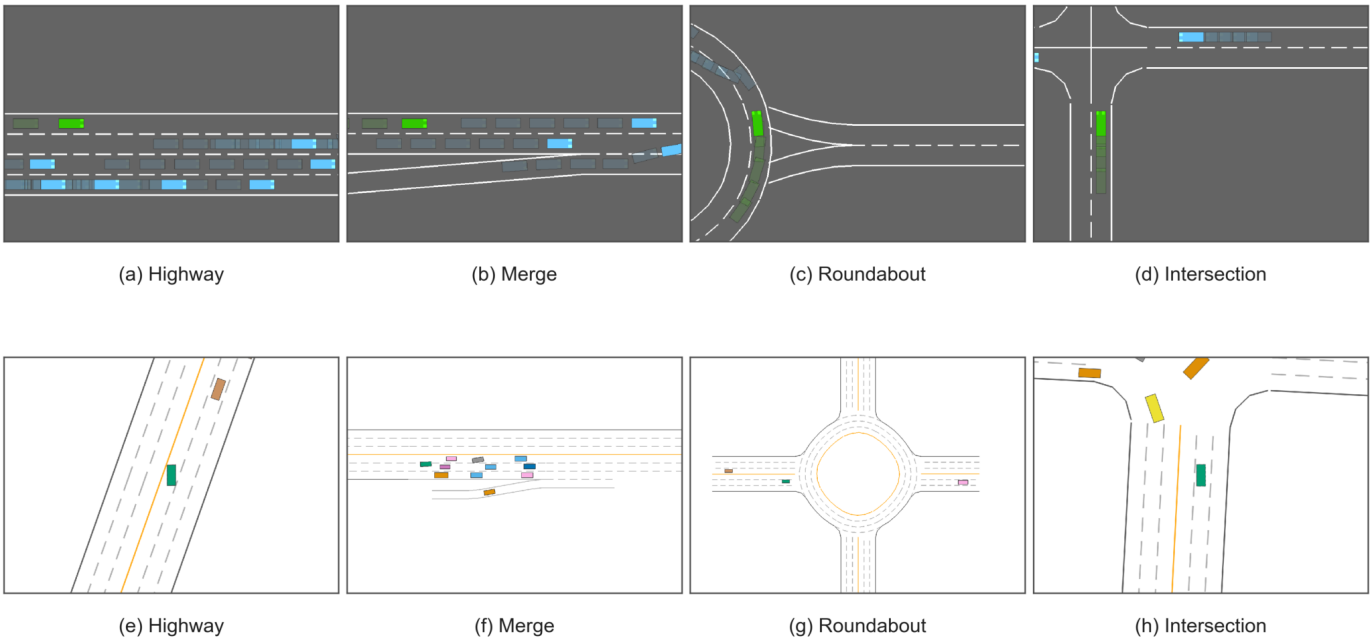

Fig. 2. Representative scenes from highway-env (top) and MetaDrive (bottom).

$\mathcal{D}(s)$. The matched-rollout utility $J_H(s,a)$ for horizon $H \in \mathbb{N}_+$ and discount $\gamma \in [0,1]$ is defined as

$$J_H(s,a) = \frac{\sum_{k=0}^{H-1} \gamma^k r_k}{\sum_{k=0}^{H-1} \gamma^k} - \mathbb{I}[E_C], \tag{7}$$

where $s_0 = s$, $a_0 = a$, the fast policy supplies a_k for $k \geq 1$, r_k is the normalized per-step simulator reward, and E_C is the event that a collision occurs within the rollout horizon. A collision terminates the rollout and incurs a unit penalty. Subsequent reward increments are zero while the denominator remains the fixed H-step discount sum. We set $H = 20$ and $\gamma = 0.99$. Candidates sharing the same projected command and target speed are merged prior to evaluation. The Recoverable Value of Deliberation (R-VoD) is then defined as

$$\text{R-VoD}(s) = \max_{a \in \mathcal{C}(s)} \left[J_H(s,a) - J_H(s, a^f_{\text{eff}}) \right]. \tag{8}$$

Since $a^f_{\text{eff}} \in \mathcal{C}(s)$, R-VoD is nonnegative by construction and equals zero when no alternative improves upon the fast continuation. A state is labeled *corrective* when $\text{R-VoD}(s) \geq \epsilon_{\text{R}}$, where $\epsilon_{\text{R}} = 0.02$ is a fixed utility-scale margin. It is independent of the online cost-scale margin δ, despite their equal numerical values. R-VoD does not enter the online gate. By varying only the first effective action while holding the remainder of the trajectory under the common fast policy, R-VoD isolates the corrective opportunity at release from downstream variation in the slow-reasoner response. This construction directly supports the corrective-state ablation and the matched-proposal release analysis reported in Section IV.

IV. Experiments and Discussion

A. Experimental Setup

RGD is evaluated on the highway-env [28] and MetaDrive [29] closed-loop simulators, with Qwen3-8B [30] as the default slow reasoner. The main highway-env experiments run at 10 Hz with 30-s episodes. Within each comparison, methods share the seed cohort, control interface, reasoner service, request budget, and evaluation rule. The paired allocation study uses 30 seeds, and the mechanism analyses use independent 20-seed cohorts.

Simulator seeds define the paired experimental units. We report 95% percentile bootstrap intervals and use exact McNemar or paired sign-flip tests, with Holm adjustment for prespecified contrasts. Gate criteria, corrective margins, and service settings are calibrated before evaluation and then frozen. R-VoD is computed offline from matched first-action rollouts. The paired cohort supplies the primary resource and task comparison, while controlled-delay and matched-proposal cohorts isolate release behavior. Fig. 2 shows representative highway-env and MetaDrive scenes covering lane following, merges, roundabouts, and intersections under the same query and release interface.

B. Performance Context Under Published Settings

Under the environment settings of PADriver [31], Table I places RGD within the operating ranges of the reported methods. RGD completes 29 of 30 runs at 611 m and 73.60 km/h, without dedicated training on the evaluation environment.

TABLE I
HIGHWAY-ENV PERFORMANCE UNDER THE PADRIVER EXPERIMENTAL CONFIGURATION.

Method	Dist. (m)	Speed (km/h)	Saf.	Kep.	Succ. (/30)
Driving with LLMs	411	49.42	0.49	0.56	20
GPT-Driver	428	51.40	0.45	0.52	23
DiLu	386	46.29	0.60	0.72	28
PADriver–Normal	603	72.47	**0.91**	**0.92**	25
RGD (ours)	**611**	**73.60**	0.75	0.86	**29**

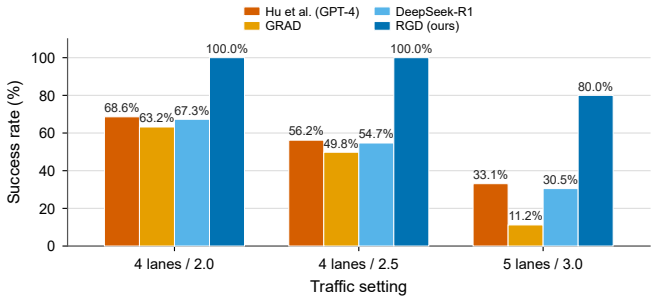

Fig. 3. Success rates obtained under the highway-env traffic settings of Hu et al. [32].

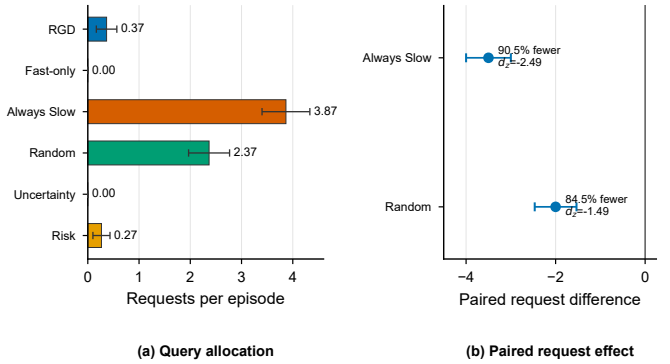

Fig. 4. Slow-path allocation under the 30-seed paired protocol. (a) Mean requests per episode with 95% seed-bootstrap intervals. (b) Paired RGD-minus-baseline differences for the two request-intensive allocators. Negative values denote fewer RGD requests.

TABLE II
RGD OPERATING METRICS UNDER THE SETTINGS OF HU ET AL.

Setting	Dist. (m)	Speed (m/s)	Req./ep.	Time (ms/frame)
4L / 2.0	69.29	23.84	1.0	58.9
4L / 2.5	66.19	22.76	1.0	65.2
5L / 3.0	52.38	17.38	0.8	50.1

On our evaluation platform, RGD operates at 37.6 ms per frame. Following the source definitions, Saf. denotes safe-distance- keeping rate and Kep. denotes lane-keep rate. Both are reported in $[0, 1]$, with larger values indicating a higher frequency of the corresponding event. RGD achieves the best distance, speed, and completion in this comparison. Relative to PADriver–Normal, it increases distance by 8 m, speed by 1.13 km/h, and successful runs by four. PADriver records the highest Saf. and Kep. values under its dedicated personalized policy objective.

Following Hu et al. [32], we evaluate RGD under the same highway-env action space, 30-frame collision-free success criterion, and 4L/2.0, 4L/2.5, and 5L/3.0 traffic configurations without added delay. Fig. 3 compares RGD with ChatGPT-4, GRAD, and DeepSeek-R1-7B under these settings. RGD reaches 100.0%, 100.0%, and 80.0%, exceeding the strongest published value in the three settings by 31.4, 43.8, and 46.9 percentage points, respectively. Table II shows that this gain is obtained with only 1.0, 1.0, and 0.8 requests per episode and a per-frame runtime below 66 ms. The largest margin occurs at 5L/3.0, where increased lane count and traffic density make query-state actions more likely to lose support before release. RGD addresses this condition at both boundaries: conjunctive admission reserves slow reasoning for supported alternatives, and current-state validation rejects a returned maneuver after its gap has closed or its advantage has disappeared. Fast-path control covers the intervals between these sparse requests. This release-aware division explains why RGD retains a large success-rate advantage as the published baselines degrade with traffic complexity. Table II reports safety-qualified speed with unsuccessful RGD runs assigned zero and the corresponding request rate for each setting.

C. Query- and Release-Side Mechanism Evidence

1) Query-Side Allocation and Delay Sensitivity: The paired allocation study compares RGD against five scheduling baselines under a common 30-seed protocol: Fast-only (never invokes the slow reasoner), Always Slow (issues a request at every eligible frame), Random (requests with uniform probability at each eligible frame), Uncertainty (requests when the fast policy entropy exceeds a threshold), and Risk (requests when a dedicated TTC-urgency score exceeds a threshold). All methods share the same Qwen3-8B service, fast controller, action mapper, release interface, and episode realization. Only the slow-path allocation policy differs. Fig. 4(a) reports requests per episode, and panel (b) reports the paired RGD-minus-baseline effects for the two request-intensive allocators.

RGD averages 0.37 requests per episode, reducing slow-path use by 90.5% relative to Always Slow and by 84.5% relative to Random. The paired differences are -3.50 requests [95% CI -4.00, -3.00] and -2.00 requests [95% CI -2.47, -1.53], with paired standardized effects $d_z = -2.49$ and -1.49, respectively. Both contrasts remain significant after Holm correction ($p_{\mathrm{H}} = 0.00025$). This corresponds to 11 RGD requests over the 30 episodes, compared with approximately 116 for Always Slow and 71 for Random. Fast-only and Uncertainty provide zero-request references, and Risk issues 0.27 requests per episode. RGD therefore occupies a sparse but nonzero allocation regime: the conjunction in (4) rejects routine, unsupported, or delay-infeasible opportunities, yet retains access to slow reasoning when all admission predicates hold. Because the reasoner service and release interface are shared, the large paired reductions are attributable to this admission policy rather than a change in model or action space. In particular, the 11 admitted requests identify the small subset of states in which delay feasibility, alternative-action support, cost advantage, and scene demand coincide. This is the selective-computation value that neither continual invocation nor a single-trigger allocator provides.

To examine the same admission mechanism when the delay constraint is relaxed, the zero-delay cohort holds the fast controller, slow reasoner, request budget, and feasibility interface fixed across all six allocation policies. The subsequent delay

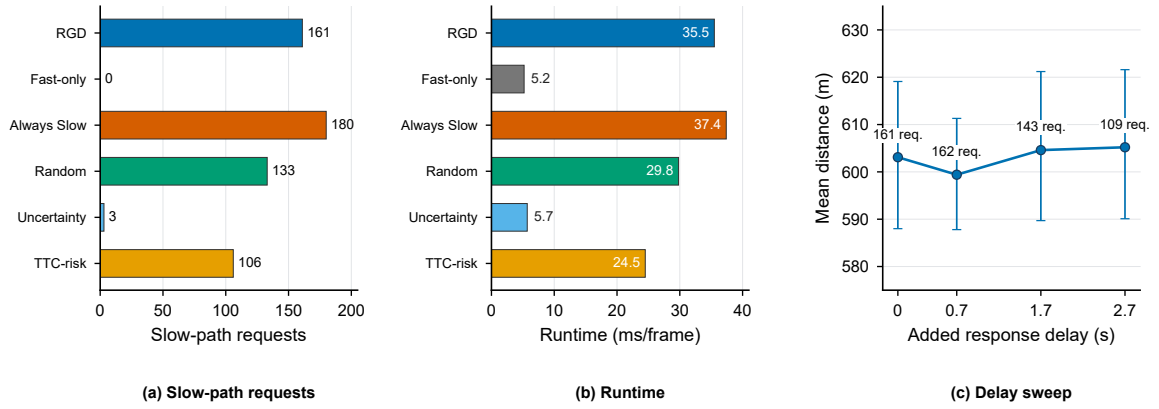

Fig. 5. Slow-path allocation under zero added delay and the fixed-gate delay sweep. (a) Total requests and (b) mean per-frame runtime in the zero-delay cohort. (c) Mean distance with 95% seed-bootstrap intervals. Labels give the corresponding RGD request counts.

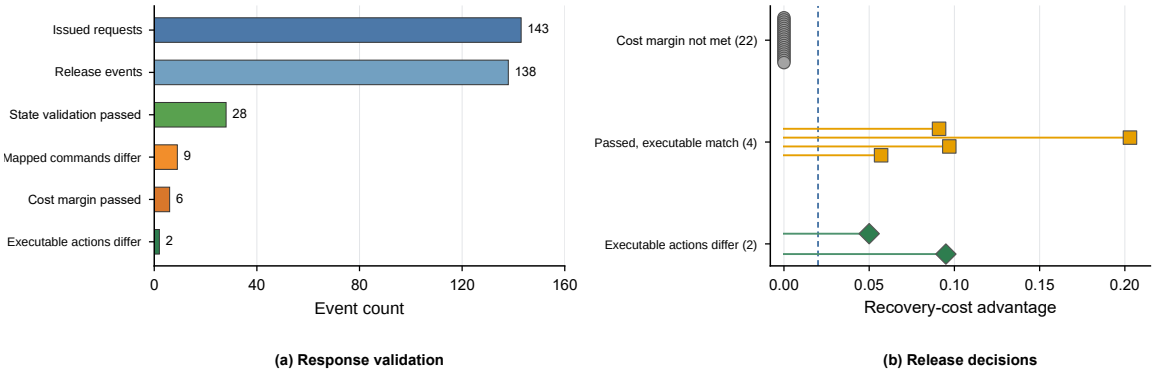

Fig. 6. Response validation under 1.7-s added delay. (a) Stage-wise retention from request issuance to post-projection actuation. (b) The 28 state-valid outcomes stratified by recovery-cost advantage. The dashed line is the fixed 0.02 margin, and "executable match" denotes equality with the projected fast action.

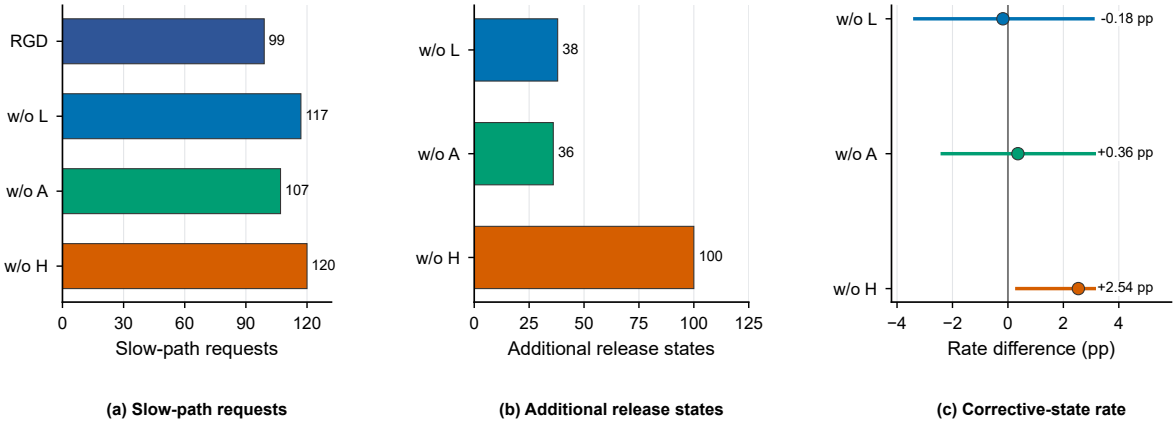

Fig. 7. One-at-a-time admission-criterion ablation over 20 seeds. (a) Total requests, (b) additional release states relative to Full RGD, and (c) R-VoD corrective-state-rate differences with 95% seed-bootstrap intervals.

sweep uses the same seeds and opportunity schedule to assess delay-feasibility effects on slow-path allocation.

Without added delay, Fig. 5(a) and (b) show that RGD schedules 161 requests, 10.6% fewer than Always Slow, and reduces mean per-frame runtime from 37.4 to 35.5 ms. Since the delay constraint is permissive in this setting, the reduction comes from the action-support, recovery-cost, and scene-demand predicates. Across the fixed-gate sweep in Fig. 5(c), requests change from 161 and 162 at 0 and 0.7 s to 143 and 109 at 1.7 and 2.7 s. The 32.3% reduction from 0 to 2.7 s occurs while mean distance remains approximately 599 to 605 m and the 95% intervals overlap. The near-equal counts at 0 and 0.7 s show that short delays leave most admitted maneuver windows intact. The reductions at 1.7 and 2.7 s begin when predicted release approaches those windows. Thus, L_t converts the predicted response interval into workload

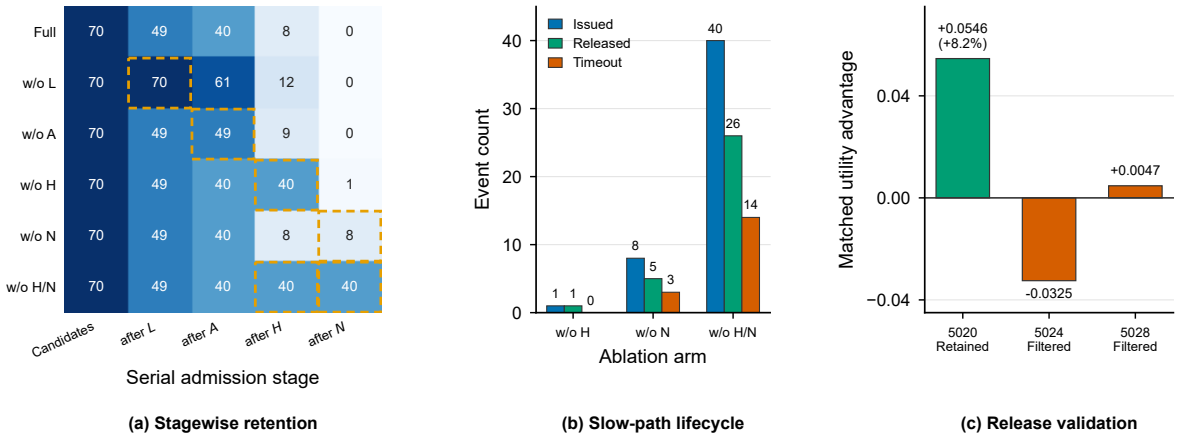

Fig. 8. Six-condition gate ablation and matched-proposal release audit. (a) Serial candidate retention. Dashed cells mark omitted predicates. (b) Issued, released, and timed-out requests over 20 paired seeds. (c) Matched-rollout utility advantage for the three post-projection distinct interventions. Horizontal labels are seed identifiers, and retained and filtered denote the release-validation outcome.

control: RGD avoids launching proposals whose supported maneuver is unlikely to remain available, without per-delay retuning while preserving the observed distance profile.

2) Release-Side Validation Under Delay: The release-side study introduces a fixed 1.7-s added delay to examine how RGD processes a slow response after the traffic state has evolved. The fast controller, slow reasoner, request budget, seed cohort, and feasibility interface remain fixed. Fig. 6 reports the response-validation stages and the resulting release decisions. Each returned proposal arrives in a state later than the state that initiated the request, and RGD applies it only after all release checks pass. Response traces follow each request from admission through response validation, current-state mapping, and actuation. The one-at-a-time ablation variants draw from a separate 20-seed cohort. Each removes one admission criterion while retaining the release procedure.

Under the 1.7-s delay, query admission issues 143 requests, of which 138 produce release events before episode termination. Current-state feasibility, support, and admissible-cost checks retain 28. Mapped-command distinctness retains nine, and the fixed recovery-cost margin retains six. The common projection Φ maps four of these six commands to the same executable action as Fast, leaving only two actions that can change actuation. Fig. 6(b) gives the same decomposition over all 28 state-valid outcomes: 22 do not meet the 0.02 margin, four pass but match Fast after projection, whereas two remain distinct at the executable-action level. The chain $143 \to 138 \to 28 \to 9 \to 6 \to 2$ is the central release result. A valid response never acquires authority by arrival alone. Of the 143 issued requests, 141 leave the final fast-path action unchanged. The trace therefore exposes the main value of the release gate: it converts abundant model output into rare actuation changes, while preserving the fast command whenever the delayed proposal is stale, redundant, or insufficiently advantageous.

Fig. 7 separates the three admission predicates under natural request flow. Relative to Full RGD's 99 requests, removing L, A, or H raises the totals to 117, 107, and 120 and exposes 38, 36, and 100 additional release states, respectively. The R-VoD corrective-state-rate changes are -0.18, $+0.36$, and $+2.54$ percentage points. The intervals for L and A straddle zero, showing that these predicates primarily regulate delay feasibility and supported-action volume. The H interval lies above zero, but the increase occurs together with 100 additional release states. Thus, H is the dominant online workload regulator in this ablation: removing it admits a much broader state population, including more states labeled corrective by the separate offline diagnostic. The positive rate difference does not mean that H enriches the selected cohort. It shows that removing H expands both request volume and corrective-state exposure. This direction separates the two roles: online recovery cost controls service load, whereas R-VoD measures the corrective opportunity that remains at release. Full RGD uses the former for selectivity and the latter for independent mechanism audit.

To complement the natural-flow ablation, we use a shared proposal bank to isolate gate and projection factors independently of scheduling variation. Six conditions are evaluated: Full RGD, removal of each of L, A, H, and N, and joint removal of H and N. All conditions replay the same gate-independent proposal source over 20 held-out seeds at a fixed 1.7-s delay. The action mapper, release rule, safety projection, budget, and cooldown remain unchanged, so each comparison removes only the named admission predicate.

Fig. 8(a) traces the 70 synchronized candidates through the serial admission stages. Full RGD retains 49 candidates after L, 40 after A, eight after H, and none after N. In sequence, L removes 30.0% of the bank, A removes 18.4% of the remaining candidates, H removes 80.0%, and N removes the final eight. Removing L or A changes the upstream trajectory but still ends at zero because the downstream predicates remain active. By contrast, removing H, N, or both leaves 1, 8, and 40 complete-gate candidates. Panel (b) converts this retention into service load: the three arms issue 1, 8, and 40 requests, of which 1, 5, and 26 reach release and 0, 3, and 14 time out. The H/N combination therefore prevents a 40-request burst from reaching the executor. Removing N and H/N also yields 4 and 16 mapped proposals distinct from Fast,

TABLE III
RESULTS WITH DIFFERENT SLOW REASONERS.

Reasoner	Valid Resp.	Distance (m)	Completion (/30)
Qwen3-8B	**138**	599.16	**29**
Qwen3.5-4B	**138**	599.16	**29**
Qwen2.5-7B	122	599.12	**29**
GPT-5.6-sol	112	**603.39**	**29**
GPT-5.6-terra	132	599.16	**29**
GPT-5.6-luna	133	599.16	**29**
Fable-5	137	599.16	**29**

TABLE IV
RESULTS ACROSS LANE AND DENSITY SETTINGS.

Lanes/Density	Distance (m)	Successful Runs (/30)
4L/2.0	599	29
4L/3.0	478	23
5L/2.0	**629**	**30**
5L/3.0	429	20
6L/2.0	611	28
6L/3.0	453	20

TABLE V
COMPOUND-HAZARD EVALUATION IN METADRIVE. A, PEDESTRIAN CROSSING. B, UNCONTROLLED INTERSECTION. C, INTEGRATED TRAFFIC.

Method	Scene	Safety (%)	Speed (m/s)
ASR-RL [7]	A	99.2	**7.1**
	B	**98.3**	**8.6**
	C	**97.9**	6.8
RGD (ours)	A	**100.0**	4.8
	B	94.8	7.0
	C	84.6	**6.9**

but the shared projection maps all of them to fast-equivalent executable actions, and task endpoints remain invariant across 120 observations organized into 20 paired seed blocks. The full gate therefore removes workload that produces no distinct actuation in this frozen bank.

The matched-proposal release analysis replays the 11 natural RGD proposals through the current release gate. Both release-validated and no-validation conditions obtain nine releases. Without validation, three returned commands remain distinct after the common projection. Release validation retains one. Fig. 8(c) reports their matched utility. The retained intervention has a $+0.0546$ utility advantage: it improves utility (7) from 0.667 to 0.721, an 8.2% relative gain, and increases 20-step progress by 23.70 m without a collision in either branch. The two filtered actions have utility advantages of -0.0325 and 0.0047, both below the fixed 0.02 corrective margin. The release gate therefore selects the only intervention with material matched-rollout value and removes one harmful and one negligible alternative. Panels (a), (b), and (c) establish a complete mechanism chain: admission controls service exposure, projection removes executable redundancy, and release validation reserves authority for the proposal with corrective value.

D. Reasoner-Interface Evaluation

On a separate 30-episode substitution cohort, the slow reasoner is replaced with six alternative models while the RGD gate, action schema, and fallback path remain unchanged. Table III shows that valid responses vary from 112 to 138, a 23% range relative to the minimum, whereas all seven models complete 29 of 30 runs. Mean distance spans only 4.27 m, from 599.12 to 603.39 m. The two Qwen3 variants return the most valid responses, while GPT-5.6-sol returns the fewest but attains the best mean distance. Response yield is therefore not a proxy for closed-loop progress under RGD.

The proposal mapper normalizes model phrasing before release validation, and an authorized slow output then joins the fast default at the shared mapper. Fallback absorbs missing, rejected, or redundant outputs. These mechanisms limit how response-yield variation reaches actuation, which explains the narrow distance range and common 29/30 completion result in Table III. The result demonstrates the value of separating reasoning from authority: model substitution changes response yield, but the release checks, shared mapper, and fast fallback limit its effect on closed-loop behavior. RGD therefore does not depend on a single slow model in the evaluated reasoner set.

E. Evaluation Across Lane and Density Conditions

Table IV reports mean distance and successful runs across six lane/density configurations without retuning the gate. At low density (2.0), successful runs range from 28 to 30 and mean distance from 599 to 629 m. At high density (3.0), successful runs range from 20 to 23, with mean distance from 429 to 478 m, as dense traffic closes feasible gaps and increases interaction cost. RGD nevertheless uses the same admission and release parameters across all six settings.

The best result occurs at 5L/2.0, with 629 m and 30 successful runs. This setting provides more lateral alternatives than 4L/2.0 while retaining the release-time gaps that high density removes. The resulting combination favors both the action-support predicate A_t and a positive recovery-cost margin. At density 3.0, the same predicates withhold authority as gaps close, and the fast path continues control instead of forcing a delayed maneuver. The table therefore shows the intended fixed-gate behavior across changing maneuver space and interaction pressure, with moderate lane multiplicity and density providing the strongest operating condition.

F. Cross-Scenario Evaluation

Following the scenario and metric definitions of Wang et al. [7], Table V compares collision-free safety and mean speed for pedestrian crossings (A), uncontrolled intersections (B), and combined hazards (C). RGD attains the highest safety in Scene A at 100%, maintains 94.8% safety at 7.0 m/s in Scene B, and records the highest mean speed in Scene C at 6.9 m/s. These operating points show a favorable balance between safety and progress under one gate configuration rather than single-metric optimization.

The balance follows the complementary gate roles. Current-state remapping suppresses delayed commands invalidated by pedestrian motion, favoring collision-free operation in Scene A. Under combined hazards in Scene C, conjunctive admission retains supported recovery maneuvers, release

comparison removes stale or redundant proposals, and the fast path preserves progress. Scene B shows that the same mechanisms jointly maintain safety and speed. RGD thus adapts its operating balance to hazard composition without scene-specific training or retuning.

G. Discussion

RGD resolves release-time mismatch by treating slow reasoning as a release-conditioned proposal. The query gate concentrates deliberation on states with supported corrective opportunities, while release validation remaps each response in the current state and admits only feasible, distinct, and advantageous proposals. Fast fallback preserves continuous control when a response is stale or redundant. The shared action interface and R-VoD diagnostic keep the authority contract and corrective-opportunity assessment consistent across reasoners and traffic conditions.

Limitations and future work. The evaluation uses discrete commands, structured outputs, and controlled simulator delays. Future work will extend the contract to continuous trajectories, stochastic latency, hardware-in-the-loop, and real-vehicle settings, with adaptive margins under distribution shift.

V. Conclusion

This paper presents RGD-Driver, a two-stage Release-Aware Gated Deliberation framework that governs slow reasoning invocation and delayed proposal release in closed-loop control. Separating query admission from release validation ensures that a delayed proposal reaches actuation only when it remains feasible, differs from the current fast action, and offers a recovery-cost advantage in the evolved traffic state. The companion R-VoD diagnostic provides an offline measure of the corrective opportunity available at release.

Comprehensive closed-loop experiments demonstrate that RGD substantially reduces slow-path invocations while preserving task outcomes, and that release validation retains the corrective intervention in the evaluated proposal stream while filtering harmful or negligible alternatives. The common action interface remained effective across slow reasoners and diverse traffic scenarios. By reconceiving slow reasoning as an expiring proposal rather than a persistent command source, RGD establishes a principled, release-aware decision interface between deliberative reasoning and real-time control.

References

[1] L. Wen *et al.*, "DiLu: A knowledge-driven approach to autonomous driving with large language models," in *Proc. Int. Conf. Learn. Represent.*, 2024.

[2] L. Chen *et al.*, "Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 14 093–14 100.

[3] J. Mao, Y. Qian, J. Ye, H. Zhao, and Y. Wang, "GPT-Driver: Learning to drive with GPT," 2023, arXiv:2310.01415.

[4] D. Pei, J. He, K. Liu, Y. Wu, Y. Lei, and X. Xiao, "Enhancement of large language models driving knowledge for practical autonomous driving decision making," *IEEE Trans. Veh. Technol.*, vol. 75, no. 8, pp. 15 944–15 959, 2026.

[5] H. Shao *et al.*, "LMDrive: Closed-loop end-to-end driving with large language models," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15 120–15 130.

[6] Z. Cui *et al.*, "C-TRAIL: A commonsense world framework for trajectory planning in autonomous driving," *IEEE Trans. Veh. Technol.*, pp. 1–14, 2026.

[7] J. Wang, H. Ren, X. Zhu, and Z. Ma, "Enhancing autonomous vehicle decision-making through policy transfer with large language model," *IEEE Trans. Intell. Transp. Syst.*, pp. 1–10, 2025.

[8] X. Wang, Z. Zhu, G. Huang, X. Chen, J. Zhu, and J. Lu, "DriveDreamer: Towards real-world-drive world models for autonomous driving," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 55–72.

[9] C. Min *et al.*, "DriveWorld: 4D pre-trained scene understanding via world models for autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15 522–15 533.

[10] W. Zheng, R. Song, X. Guo, C. Zhang, and L. Chen, "GenAD: Generative end-to-end autonomous driving," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 87–104.

[11] B. Liao *et al.*, "DiffusionDrive: Truncated diffusion model for end-to-end autonomous driving," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2025, pp. 12 037–12 047.

[12] K. Qian *et al.*, "FASIONAD: Fast and slow fusion thinking systems for human-like autonomous driving with adaptive feedback," 2024, arXiv:2411.18013.

[13] R. Zhang *et al.*, "AdaDrive: Self-adaptive slow-fast system for language-grounded autonomous driving," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 5112–5121.

[14] R. Xin *et al.*, "NetRoller: Interfacing general and specialized models for end-to-end autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 75, no. 5, pp. 7469–7482, 2026.

[15] J. Mei *et al.*, "Continuously learning, adapting, and improving: A dual-process approach to autonomous driving," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 123 261–123 290.

[16] S. Russell and E. Wefald, "Principles of metareasoning," *Artif. Intell.*, vol. 49, no. 1–3, pp. 361–395, 1991.

[17] M. Boddy and T. L. Dean, "Deliberation scheduling for problem solving in time-constrained environments," *Artif. Intell.*, vol. 67, no. 2, pp. 245–285, 1994.

[18] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI Mag.*, vol. 17, no. 3, pp. 73–83, 1996.

[19] B. Cserna, W. Ruml, and J. Frank, "Planning time to think: Metareasoning for on-line planning with durative actions," in *Proc. Int. Conf. Autom. Plan. Sched.*, 2017, pp. 56–60.

[20] A. Elboher, A. Bensoussan, E. Karpas, W. Ruml, S. S. Shperberg, and E. Shimony, "A formal metareasoning model of concurrent planning and execution," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 12 427–12 435.

[21] Y. Chen, Z.-h. Ding, Z. Wang, Y. Wang, L. Zhang, and S. Liu, "Asynchronous large language model enhanced planner for autonomous driving," in *Proc. Eur. Conf. Comput. Vis.*, 2025, pp. 22–38.

[22] S. Chen, Y. Sun, D. Li, Q. Wang, Q. Hao, and J. Sifakis, "Runtime safety assurance for learning-enabled control of autonomous driving vehicles," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2022, pp. 8978–8984.

[23] P. Tabuada, "Event-triggered real-time scheduling of stabilizing control tasks," *IEEE Trans. Autom. Control*, vol. 52, no. 9, pp. 1680–1685, 2007.

[24] M. Li, J. Gao, L. Zhao, and X. Shen, "Adaptive computing scheduling for edge-assisted autonomous driving," *IEEE Trans. Veh. Technol.*, vol. 70, no. 6, pp. 5318–5331, 2021.

[25] T. G. Molnar, A. K. Kiss, A. D. Ames, and G. Orosz, "Safety-critical control with input delay in dynamic environment," *IEEE Trans. Control Syst. Technol.*, vol. 31, no. 4, pp. 1507–1520, 2023.

[26] S. Shalev-Shwartz, S. Shammah, and A. Shashua, "On a formal model of safe and scalable self-driving cars," 2017, arXiv:1708.06374.

[27] A. Liniger and J. Lygeros, "Real-time control for autonomous racing based on viability theory," *IEEE Trans. Control Syst. Technol.*, vol. 27, no. 2, pp. 464–478, 2019.

[28] E. Leurent, "An environment for autonomous driving decision-making," 2018, GitHub repository.

[29] Q. Li, Z. Peng, L. Feng, Q. Zhang, Z. Xue, and B. Zhou, "MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 3, pp. 3461–3475, 2023.

[30] A. Yang *et al.*, "Qwen3 technical report," 2025, arXiv:2505.09388.

[31] G. Kou *et al.*, "PADriver: Towards personalized autonomous driving," in *Proc. Int. Joint Conf. Neural Netw.*, 2025, pp. 1–8.

[32] Y. Hu, D. Ou, J. Huang, M. Wu, M. Hao, and R. Yu, "Integrating vision and language foundation models for enhanced navigation and decision-making in connected autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 74, no. 10, pp. 16 233–16 249, 2025.